Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Рубежный контроль №1

«Классы данных и запросы в языке Python»

Предметная область № 32: «Колонка данных — Таблица данных»
Вариант запросов: Б
ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-33Б _____ (Ефименко К. Е.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю. Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):
- Балльная оценка:
- Оценка:

2026 г.

Цель работы: изучение способов моделирования предметных областей с использованием классов данных и реализации запросов к связанным структурам данных средствами языка Python.

Задание: разработать программу на языке Python 3 для предметной области № 32 («Колонка данных — Таблица данных») и реализовать запросы варианта Б.

В рамках рубежного контроля разработана программа на языке Python 3, реализующая модель предметной области «Колонка данных — Таблица данных» с отношениями один-ко-многим и многие-ко-многим.

В программе созданы следующие классы данных:

1) Column («Колонка данных») — сторона «много» в отношении один-ко-многим. Поля: id (числовой идентификатор), name (наименование колонки), size (размер данных в байтах — количественный признак) и table_id (внешний ключ для связи с таблицей данных);

2) Table («Таблица данных») — сторона «один» в отношении один-ко-многим. Поля: id (числовой идентификатор) и name (наименование таблицы данных);

3) TableColumn («Колонки таблицы данных») — связующий класс для реализации отношения многие-ко-многим с полями table_id и column_id.

Для проверки работы программы сформированы тестовые наборы объектов классов с согласованными идентификаторами.

В соответствии с вариантом Б реализованы три запроса:

• Запрос Б1 (один-ко-многим): вывести список всех связанных колонок данных и таблиц данных, отсортированный по колонкам данных (по наименованию колонки), сортировка по таблицам данных произвольная;

• Запрос Б2 (один-ко-многим): вывести список таблиц данных с количеством колонок данных в каждой таблице, отсортированный по количеству колонок данных (по убыванию);

• Запрос Б3 (многие-ко-многим): вывести список всех колонок данных, у которых наименование заканчивается на «ов», и названия их таблиц данных.

При реализации запросов использованы функциональные возможности языка Python: генераторы списков (list comprehensions), функции высших порядков (filter, sorted), лямбда-функции (lambda) и оператор itemgetter.

Листинг программы

Файл main.py

```
# используется для сортировки
from operator import itemgetter
```

```
class Column:
    def __init__(self, id, name, size, table_id):
        self.id = id
        self.name = name
        self.size = size
        self.table_id = table_id


class Table:
    def __init__(self, id, name):
        self.id = id
        self.name = name


class TableColumn:
    def __init__(self, table_id, column_id):
        self.table_id = table_id
        self.column_id = column_id


# Таблицы данных
tables = [
    Table(1, 'Пользователи'),
    Table(2, 'Заказы'),
    Table(3, 'Товары'),
    Table(4, 'Аналитика'),
    Table(5, 'Логи'),
]

# Колонки данных
columns = [
    Column(1, 'Счетчик просмотров', 8, 4),
    Column(2, 'Количество заказов', 4, 1),
    Column(3, 'Набор тегов', 64, 3),
    Column(4, 'Список файлов', 128, 5),
    Column(5, 'Идентификатор', 8, 1),
    Column(6, 'Электронная почта', 64, 1),
    Column(7, 'Стоимость', 8, 2),
    Column(8, 'Дата создания', 8, 2),
]
```

```
# Связь многие-ко-многим
tables_columns = [
    TableColumn(1, 2),
    TableColumn(1, 3),
    TableColumn(1, 5),
    TableColumn(1, 6),
    TableColumn(1, 8),
    TableColumn(2, 2),
    TableColumn(2, 4),
    TableColumn(2, 5),
    TableColumn(2, 7),
    TableColumn(2, 8),
    TableColumn(3, 1),
    TableColumn(3, 3),
    TableColumn(3, 5),
    TableColumn(3, 7),
    TableColumn(3, 8),
    TableColumn(4, 1),
    TableColumn(4, 5),
    TableColumn(4, 8),
    TableColumn(5, 1),
    TableColumn(5, 4),
    TableColumn(5, 5),
    TableColumn(5, 8),
]


def main():
    # Соединение данных один-ко-многим
    one_to_many = [
        (c.name, c.size, t.name)
        for t in tables
        for c in columns
        if c.table_id == t.id
    ]

    # Соединение данных многие-ко-многим
    many_to_many_temp = [
        (t.name, tc.table_id, tc.column_id)
        for t in tables
        for tc in tables_columns
```

```
        if t.id == tc.table_id
    ]

    many_to_many = [
        (c.name, c.size, table_name)
        for table_name, table_id, column_id in many_to_many_temp
        for c in columns
        if c.id == column_id
    ]

    print('Задание Б1')
    res_1 = sorted(
        [(c_name, c_size, t_name) for c_name, c_size, t_name in one_to_many],
        key=itemgetter(0)
    )
    print(res_1)

    print('\nЗадание Б2')
    res_2_unsorted = []
    for t in tables:
        t_cols = list(filter(lambda i: i[2] == t.name, one_to_many))
        if len(t_cols) > 0:
            res_2_unsorted.append((t.name, len(t_cols)))
    res_2 = sorted(res_2_unsorted, key=itemgetter(1), reverse=True)
    print(res_2)

    print('\nЗадание Б3')
    res_3 = {}
    for c in columns:
        if c.name.endswith('ов'):
            c_tables = list(filter(lambda i: i[0] == c.name, many_to_many))
            c_tables_names = [table_name for _, _, table_name in c_tables]
            res_3[c.name] = c_tables_names
    print(res_3)


if __name__ == '__main__':
    main()
```

Результаты работы программы

Результат выполнения программы main.py в консоли:

```
Задание Б1
```

```
[('Дата создания', 8, 'Заказы'), ('Идентификатор', 8, 'Пользователи'),
('Количество заказов', 4, 'Пользователи'), ('Набор тегов', 64, 'Товары'), ('Список
файлов', 128, 'Логи'), ('Стоимость', 8, 'Заказы'), ('Счетчик просмотров', 8,
'Аналитика'), ('Электронная почта', 64, 'Пользователи')]

Задание Б2
[('Пользователи', 3), ('Заказы', 2), ('Товары', 1), ('Аналитика', 1), ('Логи', 1)]

Задание Б3
{'Счетчик просмотров': ['Товары', 'Аналитика', 'Логи'], 'Количество заказов':
['Пользователи', 'Заказы'], 'Набор тегов': ['Пользователи', 'Товары'], 'Список
файлов': ['Заказы', 'Логи']}
```

Вывод: В ходе выполнения рубежного контроля №1 были закреплены практические навыки моделирования предметных областей в языке Python с использованием классов данных и организации связей один-ко-многим и многие-ко-многим. Были успешно реализованы выборки, агрегация и фильтрация с применением функциональных возможностей Python: генераторов списков, функций высших порядков filter и sorted, анонимных lambda-функций и оператора itemgetter. Разработанная программа корректно выполняет все поставленные запросы варианта Б для предметной области «Колонка данных — Таблица данных».